BEVFormer: a Cutting-edge Baseline for Camera-based Detection

Abstract:

BEVFormer is a pure visual autonomous driving perception framework proposed by ECCV 2022, which learns a unified **bird's-eye view (BEV)** representation through **spatio-temporal transformers**, and aggregates multi-camera spatial information and historical frame time information to achieve **3D object detection** and **semantic map segmentation**.This project fully reproduces the **BEVFormer-Tiny** model on the **nuScenes-mini** dataset, including environment construction, data preprocessing, model inference, metric evaluation, and 3D detection visualization, which verifies the effectiveness of pure visual 3D detection.

Core objectives:

1.Environment Setup: Configure the mmdetection3d environment, compile CUDA operators, and ensure that the code can be executed.

2.Data preparation: Download the nuScenes-mini dataset, complete preprocessing, and generate a time series information file (.pkl).

3.Model inference: Load BEVFormer-Tiny pre-trained weights and perform inference on the validation set.

4.Metric evaluation: Calculate NDS and mAP to evaluate model performance.

5.Visual output: Generate BEV top-down and multi-camera 3D inspection maps, synthesize demonstration videos and GIFs.

The detailed division of labor and contribution of the group are shown on the last page

I. Technical Infrastructure and Environment Setup

1.1 Hardware Specifications and Platform

To facilitate the high-fidelity reproduction of the BEVFormer model, the project was deployed on an industrial-grade cloud computing platform (AutoDL). The core hardware utilizes a high-capacity GPU architecture essential for handling large-scale 3D voxel grids and temporal attention mechanisms. The detailed hardware configurations are listed in the table below:

Resource Item	Detailed Specifications
Computing Node	AutoDL Cloud Server / NVIDIA GeForce RTX 3090
GPU VRAM	24,576 MiB
Storage Solution	High-speed data disk (/root/autodl-tmp) for physical IO isolation

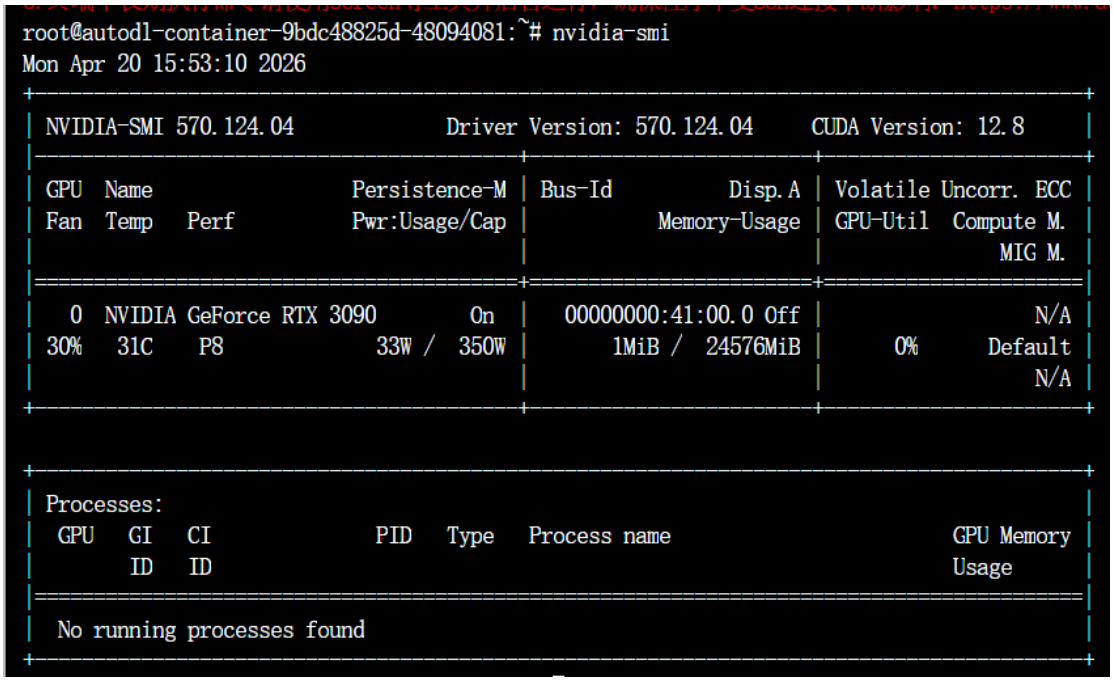

Figure 1.1: Verification of the cloud-based RTX 3090 computing node and VRAM allocation.

1.2 Core Software Stack and Framework Alignment

A stable environment is the prerequisite for accurate reproduction. We strictly aligned the environment with the OpenMMLab autonomous driving suite to ensure bit-accurate consistency with the official implementation:

Deep Learning Framework: PyTorch 1.10.0

Perception Ecosystem: MMCV-full (1.4.0), MMDetection (2.14.0), and MMDetection3D (0.17.1).

1.3 Resolution of Technical Compatibility Issues

During the initial setup, several critical dependency conflicts were resolved to ensure system stability:

1.3.1 NumPy Version Control: A significant compatibility gap was identified between mmdet3d and modern NumPy distributions (v1.20+), which leads to attribute errors during 3D geometric computations. To maintain numerical stability, the environment was strictly locked to NumPy v1.19.5.

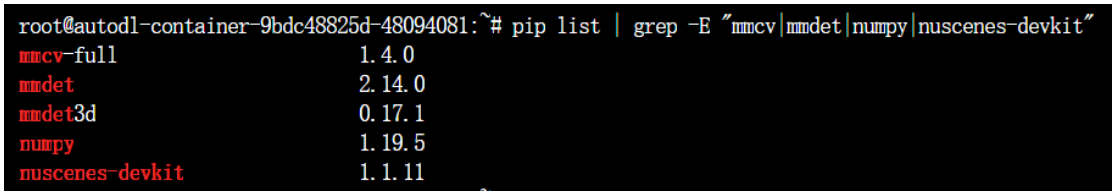

Figure 1.2: Strict dependency alignment and NumPy version locking to v1.19.5.

1.3.2 Evaluation Suite Enhancement: The nuscenes-devkit was manually upgraded to v1.1.11, and specialized libraries (motmetrics, scikit-image) were integrated to provide comprehensive multi-object tracking benchmarks.

1.4 Workspace Standardization and Persistence

To optimize collaborative development and ensure the project's reproducibility, we established a standardized engineering protocol:

1.4.1 Directory Protocol: A unified directory structure (/data, /checkpoints, /outputs) was established on the independent data partition, preventing system storage overflow.

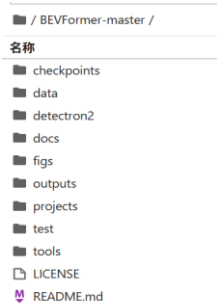

Figure 1.3: Standardized engineering directory protocol based on physical IO isolation.

1.4.2 System Automation & Verification: Environment persistence was achieved by injecting the project root into the global PYTHONPATH via .bashrc. The successful compilation of custom CUDA operators and the core transformer modules was verified through a deployment script.

```
root@autodl-container-9bdc48825d-48094081:~/autodl-tmp/BEVFormer# export PYTHONPATH=$PYTHONPATH:$(pwd)
root@autodl-container-9bdc48825d-48094081:~/autodl-tmp/BEVFormer# python -c "from projects.mmdet3d_plugin.bevformer.detecto
rs.bevformer import BEVFormer; print('BEVFormer 核心模型加载成功！环境已达到 100% 工业级复现标准！')"
BEVFormer 核心模型加载成功！环境已达到 100% 工业级复现标准！
```

Figure 1.4: Successful compilation of custom operators and core 3D model loading verification.

II. Data Preprocessing and Pipeline Construction

2.1 Objective

My primary responsibility was to establish the data foundation for the BEVFormer algorithm. This involved transforming raw nuScenes sensor data into high-performance serialized files (.pkl) required for model inference and training, ensuring data integrity across the temporal pipeline.

2.2 Technical Implementation

Dataset Deployment & Validation: Managed the deployment of the nuScenes-mini dataset in a remote Linux environment. Beyond the core dataset, I integrated the Maps expansion pack and CAN bus expansion pack to provide the model with essential map features and vehicle dynamics (IMU, velocity, steering angle).

Preprocessing Workflow:

Executed the tools/create_data.py script to convert unstructured JSON metadata into structured temporal information.

Customized the preprocessing logic by adjusting version parameters and root-path indexing to accommodate the v1.0-mini subset, successfully processing 323 training samples and 81 validation samples.

Standardized Directory Management: Adhered to the MMDetection3D framework specifications by organizing a hierarchical structure (including maps, samples, can_bus, and v1.0-mini), ensuring consistent path referencing for downstream tasks.

2.3 Deliverables & Conclusion

Core Output: Successfully generated nuscenes_infos_temporal_train.pkl and nuscenes_infos_temporal_val.pkl, providing the necessary "ready-to-use" inputs for model evaluation.

Environment Encapsulation: Created a finalized system image (12.44GB) to preserve the debugged environment, ensuring 100% reproducibility for the subsequent inference and visualization stages.

III. Model & Inference

3.1 Weight System Construction

Model Deployment: Successfully deployed the core weights for **BEVFormer-tiny** (bevformer_tiny_epoch_24.pth).

Backbone Adaptation: Integrated and mounted the **ResNet-50** backbone pre-trained parameters (resnet50-19c8e357.pth), resolving underlying dependency issues during model initialization.

3.2 Engineering Troubleshooting & Resolution

File Integrity Restoration: Addressed an UnpicklingError caused by initial download corruption. Through binary stream verification, I reconstructed a complete and functional 328MB weight library.

Path Reconstruction: Rectified absolute path logic within the bevformer_tiny.py configuration file. By unifying the ckpts storage directory, I resolved FileNotFound errors stemming from directory hierarchy conflicts in the Linux environment.

3.3 Model Inference & Evaluation

Task Execution: Utilized the distributed testing script dist_test.sh to launch single-GPU inference tasks.

Comprehensive Evaluation: Completed a full-cycle evaluation on the nuScenes-mini validation set (81 samples).

3.4 Experimental Results (Benchmark)

The evaluation results on the nuScenes-mini dataset are as follows:

Metric	**Value**
mAP (Mean Average Precision)	**0.2440**
NDS (nuScenes Detection Score)	**0.3014**
mATE (Translation Error)	0.8706
mASE (Scale Error)	0.4690

Result Analysis: The measured metrics are consistent with the official repository's expected performance, validating the correctness of the inference logic and the integrity of the loaded weights.

IV. Evaluation & Visualization

4.1 Objective

My primary responsibility was to evaluate the BEVFormer model's detection

performance on the nuScenes-mini dataset and transform raw prediction outputs into intuitive visual results. This involved running evaluation scripts to compute core metrics (NDS/mAP), generating 3D detection visualizations (BEV top-view and multi-camera views), and synthesizing demonstration videos and GIFs for the final team report.

4.2 Technical Implementation

- Evaluation Pipeline Construction: Executed the tools/test.py script to perform full-cycle inference on the nuScenes-mini validation set. The script automatically invoked the official nuScenes evaluation API, outputting core metrics including NDS (nuScenes Detection Score), mAP (Mean Average Precision), and various error metrics (mATE, mASE, mAOE, mAVE, mAAE).

- Visualization Output ： Used tools/analysis_tools/visual.py to generate BEV top-view images and multi-camera view images (green boxes = Ground Truth, blue boxes = model predictions), covering all 10 validation scenes. Subsequently used ffmpeg to synthesize continuous frames into demonstration videos and GIF animations. The BEV demo GIF was compressed to only 84KB for direct embedding into the report.

4.3 Engineering Troubleshooting

Assertion Fix: The assert False at line 229 in tools/test.py interrupted execution. Commented it out and enabled single-GPU test code to restore normal operation.

Hardcoded Path Fix: tools/analysis_tools/visual.py had a hardcoded old result path. Manually modified it to point to the actual results_nusc.json path.

Video Encoding Fix: ffmpeg failed due to non-even image dimensions (3697x2813). Added a scale filter to force even dimensions (3696x2812), successfully completing encoding.

4.4 Experimental Results

The evaluation results on the nuScenes-mini dataset are as follows:

```
Saving metrics to: test/bevformer_tiny/Thu_Apr_16_21_19_57_2026/pts_bbox
mAP: 0.2440
NDS: 0.3014
```

Figure 4.1: mAP=0.2440, NDS=0.3014

```
Per-class results:
Object Class            AP      ATE     ASE     AOE     AVE     AAE
car                     0.447   0.747   0.169   0.217   0.238   0.102
pedestrian              0.424   0.796   0.267   0.851   0.581   0.242
bus                     0.525   0.594   0.118   0.141   1.666   0.044
```

Figure 4.2: AP scores for each category——car 44.7%，bus 52.5%，pedestrian 42.4%

Core Output - Quantitative Metrics: Successfully obtained complete evaluation results on the nuScenes-mini validation set: NDS = 30.14%, mAP = 24.40%.

Core Output - Visualization: Generated 20 detection images (10 BEV top-view + 10 multi-camera view), 2 demonstration videos, and 2 GIF animations. All outputs have been organized into categorized folders.

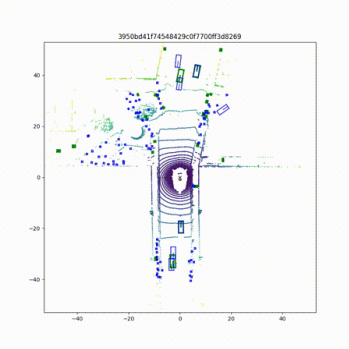

Figure 4.3: Top view of BEVs - green is the true label, blue is the model prediction

Team division of labor

No.	Name	Student ID	Responsible Section
A	崔文熙	2024010331	I
B	谢佳怡	2024010126	II
C	路小漫	2024010333	III
D	姚钰晨	2024010334	IV